

Towards a Formal Definition of Agent Memory: Basis, Span, Optimality, and the Sequential Memory Problem

Hongyao Tang

Tianjin University

tanghongyao@tju.edu.cn

Abstract. Despite the wide deployment of memory in large-model agents, there is no unified formal account of what a memory is or when it is optimal. This paper takes a first step toward this account. The central idea is that memory is a basis, knowledge is its span, and answerability is a coverage problem: an agent stores events extracted from a material; a generation operator turns any event set into the knowledge it entails; and a query is answerable exactly when some single item in the span covers it. The optimal memory is then the capacity-constrained maximizer of expected coverage, and its value traces a utility–capacity frontier, the common yardstick on which memory systems can be compared. Next, we consider noise in the memory and discuss coverage versus precision under it: a memory may store false claims, so the write policy must infer the truth of what it stores. Drawing an analogy with biological memory, which is formed continuously through ongoing experience, we formalize the continual agent-memory problem in a sequential MDP that covers multiple levels, where memory is the state, writing is the action, and the utility settled at query time is the delayed reward that drives learning. To make the framework concrete, we instantiate it on Homer’s *Odyssey*, turning the frontier, the compression zone, and the divergence of coverage from precision into concrete numbers. Finally, we position existing systems within the framework, making “how good is a memory” measurable and recasting the open problems of constructing and learning agent memory as concrete research questions.

“Tell me, Muse, of the man of many ways, who was driven far journeys . . .”
(the Muse is the daughter of Mnemosyne, the goddess of memory)
 — Homer, *The Odyssey* 1.1, trans. R. Lattimore (1965)

1 Introduction

Modern LLM agents accumulate interaction histories (dialogues, documents, observation streams) that quickly exceed the context window (Bai et al., 2024), and increasingly manage them with explicit memory modules that store, retrieve, and update over time (Packer et al., 2023; Park et al., 2023). Memory has become central to agent design, and a growing family of systems, from hierarchical page stores (Packer et al., 2023) and knowledge graphs (Jiménez Gutiérrez et al., 2024) to reflection-based reasoning (Shinn et al., 2023) and reinforcement-learned writing (Yu et al., 2026), attests to its practical importance.

The literature uses the word for conversation summaries, knowledge graphs, reflection texts, and parameter updates alike; “memory quality” is measured differently from benchmark to benchmark; and optimality is rarely even posed as a problem (Du et al., 2025). The brain sciences, by contrast, have long studied memory as a structured object: episodic and semantic memory are distinguished (Tulving, 1972), and their interaction is characterized through complementary learning systems (Kumaran et al., 2016). For large model agents, memory remains an undefined word rather than such an object. The result is a fragmented space of methods that cannot be compared on a common scale, and a field that cannot yet state its most basic questions.

- ① What is a memory for an agent, and how should it be formally defined?
- ② What makes a memory good, and what is an optimal memory?
- ③ How should a memory be written, and how should its problem settings be defined?

In this paper, we make our preliminary attempt to answer the three questions above by developing a single formal framework for agent memory. The framework supplies the three things the field lacks, one for each question: a formal definition of what a memory is, a measure of when a memory is optimal, and a formulation of how a memory is written.

We begin with the definition. It rests on a two-layer separation: an agent stores *events*, atomic statements about a material, but events are not what answer queries; what answers a query is the *knowledge* that a set of events entails. On this we build the paper’s central idea, **memory is a basis, knowledge is its span, and answerability is a coverage problem**: a memory is a subset of the events extracted from a material, a generation operator Φ maps any event set to the knowledge it entails, and the span of a memory is what the agent can actually draw on. Under a single-item support principle (every query is answered by a single knowledge item), a query is answerable exactly when some spanned item covers it, so memory construction becomes a coverage problem: with at most S events, cover as much of the query distribution as possible.

With the definition in hand, what makes a memory good becomes a well-posed optimization. The optimal memory is the capacity-constrained maximizer of expected coverage, and its value traces the **utility–capacity frontier**, the best answerability rate achievable with S events. The frontier saturates at the full-context baseline and delimits the *compression zone* in which memory matters; it maps every memory system to a point in the size–utility plane, where the gap below the frontier is what a write policy loses, and it is the common yardstick on which memory systems can be compared.

The hardest of the three questions is how a memory gets written, and two features of the real setting make it hard. Extraction is noisy, so we separate **coverage from precision** and measure how much of a memory’s reported quality is bought with false claims, the *water-inflation degree*. The query distribution is unknown, so writing must be learned from queries disclosed one at a time; we organize the settings of memorization into a progressive taxonomy and unify single- and multi-material writing in a **sequential MDP**, exposing delayed reward, credit assignment, and trust estimation as the core learning problem.

To make our formal framework concrete, we present an illustrative example based on Homer’s *Odyssey*, computing the framework’s objects in full so that the frontier, the compression zone, and the divergence of coverage from precision become concrete numbers. Further, we map representative memory systems onto the framework, showing where existing methods sit and how they become comparable, and we close by distilling the problems left open into a research agenda and stating the framework’s limitations.

We make the following contributions, the first three answering the three questions above.

- **A formal definition of agent memory.** Memory is an event subset of a material, and its span is the knowledge the events generate; explicit assumptions (self-containment, monotonicity) together with single-item support make the object well-defined and push composition into the generation layer.
- **An optimality theory and a common yardstick.** The optimal memory is the capacity-constrained maximizer of a coverage utility, and its value traces the utility–capacity frontier, delimiting the compression zone in which memory matters. The underlying optimization reduces to weighted maximum coverage with a greedy $(1 - \frac{1}{e})$ -approximation when the operator is decomposable, and to monotone set-function maximization in general, where cross-event synergy defeats such guarantees.
- **Memorization under noise and over time.** We separate coverage from precision under noisy extraction, organize the settings of memorization into a progressive taxonomy, and unify single- and multi-material writing in a sequential MDP, framing writing as a learning problem of delayed reward and credit assignment.
- **A positioning of existing systems and a research agenda.** We map representative memory systems onto the framework, showing that the field’s methods differ mainly in their choices of generation, writing, reading, and reasoning components, and we turn the problems this leaves open into a concrete research agenda.

The rest of the paper is organized as follows. Section 2 formalizes events, knowledge, the generation operator, and coverage utility. Section 3 defines optimal memory and the utility–capacity frontier. Section 4 extends the definition to noisy extraction. Section 5 organizes problem settings into a progressive taxonomy and unifies

them in a sequential MDP. Section 6 gives an illustrative example, Section 7 reviews related work through the framework, and Section 8 discusses a research agenda and the framework’s limitations.

2 Defining Agent Memory: Basis and Span

The framework rests on three ingredients: what memory stores, what it entails, and how to measure whether it helps. This section fixes each in turn. Section 2.1 separates the stored surface of a material from the knowledge it entails; Section 2.2 turns knowledge into answerability and then into a utility; Section 2.3 states the assumptions on the generation operator that give the theory its structure.

2.1 Events, Knowledge, and Memory

Memory operates on raw material: documents, conversations, observation streams, and multimodal sources such as image-text articles or videos. To make memory precise we keep two layers apart. The surface layer is what an agent literally stores, *events* (atomic statements about the material); the semantic layer is what those events entail, *knowledge*. We define the two in turn.

Definition 2.1 (Events and materials). *Let E be a space of events. We use event broadly: an event $e \in E$ is an atomic statement about a material, whether a happening (a fact), a preference, a rule, or a reflection, typically as simple as a subject–predicate–object tuple. A material D is a long source of information (a text, a conversation, an observation stream, an image-text article, a video); we write $E_D \subseteq E$ for the set of events contained in D .*

Events are the unit of storage, but events are not what answer queries. A raw event (“the supplier raised prices in March”) is too low-level for most questions; what answers a query is the knowledge that a set of events supports (“why did costs rise?”). We therefore introduce a generation operator that maps any set of events to the knowledge it entails.

Definition 2.2 (Knowledge and the generation operator). *A knowledge item $n \in N$ is an atomic, self-contained unit of information. The generation operator $\Phi : 2^E \rightarrow 2^N$ maps a set of events $A \subseteq E$ to the set of knowledge they generate:*

$$n \in \Phi(A) \subseteq N \iff n \text{ is generated by } A. \quad (1)$$

A single event is a degenerate knowledge item, i.e. $E \subset N$. For a material D we write $N_D = \Phi(E_D)$ for the knowledge contained in D .

Events and the generation operator give us the raw ingredients, but the object this paper studies has not yet been named.

Definition 2.3 (Memory). *A memory of a material D is a subset of its events,*

$$M_D \subseteq E_D. \quad (2)$$

*In the basis-and-span view, the stored events form a **basis**, and the knowledge they generate,*

$$R_{M_D} = \Phi(M_D), \quad (3)$$

*is the **span** of the memory. The whole material is the maximal memory E_D , whose span is the full knowledge N_D .*

Definition 2.3 is the object this paper is about: a memory is a basis of events, and its span is what the agent can actually draw on. The material’s full knowledge $N_D = \Phi(E_D)$ is the span of the maximal memory, and any smaller memory forgoes part of it. The gap between R_{M_D} and N_D is the knowledge a memory forgoes, and its utility cost is the query mass the span leaves uncovered; keeping that uncovered mass small is the subject of the rest of the paper. Figure 1 summarizes the two-layer picture.

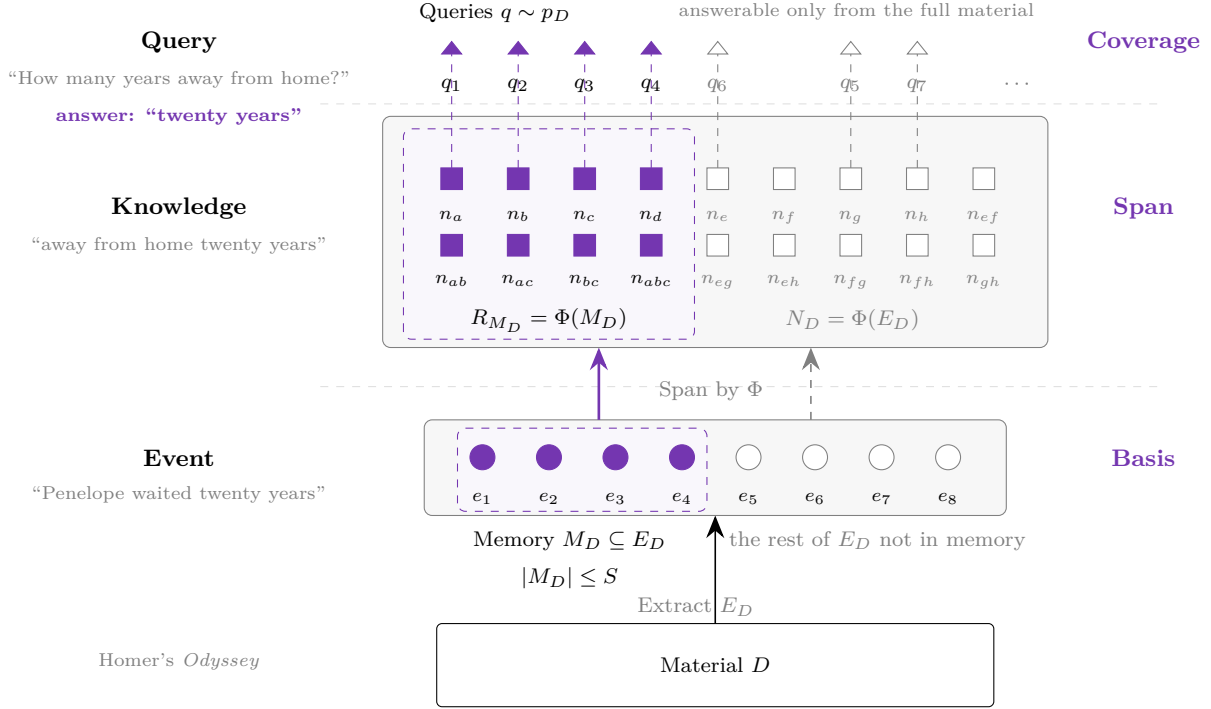


Figure 1 Memory is a basis; knowledge is its span. Extracting the events E_D from a material D and storing at most S of them forms a memory $M_D \subseteq E_D$, the dashed box inside the full event set; monotonicity of Φ then lifts the memory to a span $R_{M_D} = \Phi(M_D)$, drawn as the purple box nested inside the full knowledge $N_D = \Phi(E_D)$ (gray box): $R_{M_D} \subseteq N_D$. Under single-item support a query is answerable exactly when some spanned item covers it (dashed edges). Queries are drawn from the material’s query distribution p_D : the purple queries are covered by the memory’s span, while the gray queries $q_5, q_6, q_7, \dots$ are answerable only from the full material, the cost of compression. The item n_{ab} is spanned by e_1 and e_2 jointly: composition happens inside Φ . The left column instantiates the chain concretely on the *Odyssey*.

2.2 Query Answering with Memory

A memory is useful only insofar as it makes queries answerable. We define answerability at the level of a single knowledge item, then lift it to memories through an assumption about how items combine.

Definition 2.4 (Answerability). Let $\mathcal{Q}$ be a space of queries. The atomic relation $\text{ans}(n, q) \in \{0, 1\}$ indicates whether the single knowledge item n suffices to answer the query q . The **answerable set** of n is

$$\mathcal{Q}(n) = \{q \in \mathcal{Q} : \text{ans}(n, q) = 1\}. \quad (4)$$

Each knowledge item thus covers a set of queries, $\mathcal{Q}(n)$: the questions it alone suffices to answer. Whether a whole memory answers a query then depends on how these item-level sets combine, and here we make the central assumption of the framework.

Assumption 2.1 (Single-item support). Every query is answered by a single knowledge item: composition and reasoning do not happen at answer time but are pushed into the generation operator Φ . Equivalently, a knowledge set K answers q if and only if $q \in \bigcup_{n \in K} \mathcal{Q}(n)$.

This assumption is a design choice, and it is worth saying what it buys and what it costs. It rules out joint reasoning at answer time: the agent does not combine items on the fly to answer a query, and every chain of reasoning must already be realized inside Φ and materialized as a single knowledge item. The payoff is that answerability becomes a pure coverage question, which makes optimal memory construction a well-defined optimization problem (Section 3).

Definition 2.5 (Coverage utility). *The coverage utility of a memory M_D on a query q is*

$$u(q, M_D) = \mathbf{1}\left[q \in \bigcup_{n \in \Phi(M_D)} \mathcal{Q}(n)\right] \in \{0, 1\}, \quad (5)$$

whether the memory’s spanned knowledge contains a single item that answers q .

The utility has an explicit coverage structure: the answerable set of a memory is the union $\bigcup_{n \in \Phi(M_D)} \mathcal{Q}(n)$, so the expected utility $\mathbb{E}_{q \sim p_D}[u(q, M_D)]$ is the probability that a query drawn from the material’s query distribution is covered by the memory’s span. Optimal memory construction is therefore a coverage problem: use at most S events to cover as much query mass as possible (Section 3).

Coverage is not correctness. The utility above records only whether the necessary information is present in memory, not whether the agent produces the right answer. We keep these two concerns apart.

Definition 2.6 (End-to-end utility). *The coverage utility measures only whether necessary information is available; it does not measure whether the agent answers correctly. We separately define the **end-to-end utility***

$$u^{\text{e2e}}(q, M_D, \pi_{\text{reasoner}}) = \mathbf{1}\left[\text{Correct}(\pi_{\text{reasoner}}(q, M_D), q)\right], \quad (6)$$

where π_{reasoner} is the reasoning policy and $\text{Correct}(\cdot, q)$ judges whether an output correctly answers q . The reasoner is understood to answer from the retrieved subset of the memory’s span (Section 7). Throughout we take the proxy stance that, under noise-free memory, sufficient information approximately equals answerability, and we optimize u ; whether the reasoner makes good use of available information is treated as a separate layer.

So far the utility is defined for a single query. To optimize a memory we need to weigh queries and materials, which brings us to distributions.

Definition 2.7 (Query and material distributions). *For a material D , $p_D(q)$ denotes the distribution over queries that an agent is expected to face for D (in long interactive settings, the questions a user is likely to ask). $p(D)$ denotes a distribution over materials. Both are typically unknown and must be estimated from sequentially disclosed queries (Section 5).*

2.3 Assumptions on the Generation Operator

The theory’s structure rests on two defaults about Φ , which in practice is an abstract procedure implemented by an LLM performing extraction, induction, and summarization. We assume **self-containment** (Assumption B.1): storing an event never throws it away. We assume **monotonicity** (Assumption B.2): more memory is never worse, so coverage is non-decreasing in the stored set. Two deliberate omissions matter as much. We do *not* assume **decomposability**: cross-event composition can generate knowledge that no single event yields, which is exactly how a few events span more than the sum of their parts, and what makes the optimality problem of Section 3 harder in general. And real extraction meets these properties only approximately: an LLM is at best *locally* monotone, so injecting a conflicting fact can overturn earlier conclusions. The full statements, with the rationale and the consequences of relaxing each assumption, are given in Appendix B.

3 Optimal Memory and the Utility–Capacity Frontier

This section gives the question “what is a good memory” a precise answer. Optimality is always relative to a material D and a query distribution p_D (or, when marginalized over a material distribution $p(D)$, relative to a family of materials, Section 5). Two conventions keep the treatment honest. First, “optimal memory” is really a property of the *writing* that produces it, so we study memories $M_D = \pi_{\text{write}}(D)$ generated by a write policy. Second, we defer the representation of M_D (graphs, hierarchical pages, parameter vectors): the account here is about what to store, not how to lay it out.

3.1 Optimal Memory and Write Policy

A **write policy** turns a material into a memory, $\pi_{\text{write}}(D) \subseteq E_D$, and the **optimal memory** is any maximizer of the expected coverage utility (Definition 2.5). By monotonicity (Section 2.3) this is trivial without a

budget: storing every event attains the maximum (Proposition 3.1), so the unconstrained statement and its tied maximizers are spelled out in Appendix C. Memory matters precisely because capacity is limited, so from here on we optimize under a bound.

Definition 3.1 (Optimal memory of constrained capacity). *With a capacity bound S , the capacity-constrained optimal memory is*

$$M_D^*(S) = \arg \max_{\substack{M_D \subseteq E_D \\ |M_D| \leq S}} \mathbb{E}_{q \sim p_D} [u(q, M_D)], \quad (7)$$

*and the value it attains defines the **utility–capacity frontier***

$$U_D^*(S) = \max_{\substack{M_D \subseteq E_D \\ |M_D| \leq S}} \mathbb{E}_{q \sim p_D} [u(q, M_D)]. \quad (8)$$

*The **capacity-constrained optimal write policy** is any π_{write}^* whose output attains this optimum; the frontier value $U_D^*(S)$ is exactly the utility such a policy attains.*

Because u is a $\{0, 1\}$ indicator, the expected utility is a probability, $\mathbb{E}_{q \sim p_D} [u(q, M_D)] = \Pr_{q \sim p_D} [u(q, M_D) = 1]$: the chance that a random query is answerable from the memory’s span. The frontier $U_D^*(S)$ is thus the **query answerability rate** achievable with S events, and it is the number on which different memory systems can be compared on a common scale: a system’s memory is good insofar as it reaches a high point of the frontier at a small size.

3.2 Properties of the Frontier

The frontier inherits its shape from the generation operator. Under monotonicity, it settles into a clean picture.

Proposition 3.1 (Monotonicity of utility). *Under Assumption B.2, for any $M_D \subseteq M'_D \subseteq E_D$ and any query q , $u(q, M_D) \leq u(q, M'_D)$; hence the expected utility $M_D \mapsto \mathbb{E}_{q \sim p_D} [u(q, M_D)]$ is non-decreasing.*

Proof. If $M_D \subseteq M'_D$, then $\Phi(M_D) \subseteq \Phi(M'_D)$ by monotonicity, so the union $\bigcup_{n \in \Phi(M_D)} \mathcal{Q}(n)$ is contained in $\bigcup_{n \in \Phi(M'_D)} \mathcal{Q}(n)$, and the indicator of Definition 2.5 is non-decreasing. $\square$

Proposition 3.2 (Frontier saturation). *Under Assumption B.2, the utility–capacity frontier is non-decreasing in S , and $U_D^*(S) = \mathbb{E}_{q \sim p_D} [u(q, E_D)]$ for every $S \geq |E_D|$: with enough capacity, the trivial memory that stores everything is optimal.*

Proof. The feasible set $\{M_D \subseteq E_D : |M_D| \leq S\}$ grows with S , so the maximum is non-decreasing. For $S \geq |E_D|$ the full set E_D is feasible and, by Proposition 3.1, dominates every other memory. $\square$

Proposition 3.2 gives the definition its empirical content. It identifies a **full-context baseline**: the value $\mathbb{E}_{q \sim p_D} [u(q, E_D)]$ that an agent achieves by keeping the entire material in context. Because the frontier saturates at this value, memory genuinely matters only in the **compression zone**, the regime $S < |E_D|$ in which a few events must span knowledge as close as possible to the full-context level. The vertical gap between $U_D^*(S)$ and the baseline is exactly the cost of compression.

The frontier also gives every write policy a yardstick. A policy π produces a memory of size $|\pi(D)|$ with expected utility $\mathbb{E}_{q \sim p_D} [u(q, \pi(D))]$, so it defines a point on or below the frontier; its vertical distance from $U_D^*(|\pi(D)|)$ is the **memory-efficiency loss** of the policy. The three gaps are distinct: the **span gap** R_{M_D} vs N_D says what a memory fails to span; the compression cost is the vertical gap between the frontier and the full-context baseline, paid by optimal compression itself; the memory-efficiency loss is a policy’s vertical gap to the frontier at its own capacity. Section 6 computes $U_D^*(S)$ on a hand-checked instance and reads the compression cost and the memory-efficiency loss off it.

3.3 Solving Optimal Memory as Maximum Coverage

The capacity-constrained problem is a set-function maximization, and its tractability hinges on how Φ composes events. To see why, view each event e through the only thing that makes it useful: the set of queries its knowledge answers, which we denote

$$S_e = \bigcup_{n \in \Phi(\{e\})} \mathcal{Q}(n). \quad (9)$$

Choosing a memory of size at most S is then choosing at most S of these query sets, and the value of the choice is the query mass they cover. Whether this is the whole story depends on whether the sets combine additively: when Φ decomposes, a memory’s coverage is exactly the union of its events’ sets; when events interact, the union is enriched by knowledge that no single event yields. The following proposition characterizes these two extremes.

Proposition 3.3 (The complexity of optimal memory). *Let Φ be monotone.*

- (a) *If Φ is decomposable, $\Phi(M_D) = \bigcup_{e \in M_D} \Phi(\{e\})$, then $U_D^*(S)$ equals the value of a weighted maximum-coverage instance over the per-event query sets S_e defined above, with element weight $p_D(q)$. This optimization is NP-hard, and the greedy algorithm attains the classical $(1 - \frac{1}{e})$ -approximation.*
- (b) *In general, the objective $M_D \mapsto \mathbb{E}_{q \sim p_D}[u(q, M_D)]$ is a monotone set function (Proposition 3.1) but need not be submodular (Remark C.2): the reduction to maximum coverage fails, and no greedy-style guarantee applies.*

Proof sketch. Under decomposability the expected utility equals the total query mass covered by the chosen events, the value of a weighted maximum-coverage instance; the classical NP-hardness and $(1 - \frac{1}{e})$ greedy guarantee then apply. Part (b) follows from Proposition 3.1 together with Remark C.2; the complete argument is given in Appendix C. $\square$

Taken together, the two cases give the section’s conclusion in plain terms. With a decomposable operator, optimal memory is a coverage purchase: buy the S events that cover the most query mass, and greedy buying stays within the factor $(1 - 1/e)$ of the best possible. With interacting events, the purchase intuition survives but the guarantee does not: once marginal returns can grow, no off-the-shelf approximate algorithm applies. The tension is fundamental rather than incidental: the cross-event synergy behind a span’s power is exactly what defeats the greedy argument. This is also the first place the framework points beyond one-shot optimization: when the query distribution is itself unknown (Section 5), the objective $\mathbb{E}_{q \sim p_D}$ cannot even be evaluated, and writing must be learned sequentially rather than solved in one shot.

4 Noisy Memory: Coverage versus Precision

So far memory could contain only true events. Real extraction is noisy: the LLM that implements Φ and the extraction step may hallucinate, quote out of context, or inject claims that contradict the material. This section drops the assumption that memory is error-free, and separates two quantities that noise drives apart: how much a memory *covers* and how much of that coverage is *correct*.

4.1 Claims and the Precision Gap

Definition 4.1 (Claims). *A claim is a candidate event: an atomic statement produced by the extraction step, which may or may not be true of the material. We write $\hat{E}_D \supseteq E_D$ for the set of claims extracted from a material D , so that a memory is now any $M_D \subseteq \hat{E}_D$. The generation operator is defined on atomic statements, so it applies to claims as well as events: for any claim set, $\Phi(M_D)$ is the knowledge the stored statements would support, whether they are true or not. Each claim e carries a truth value $\tau(e) \in \{0, 1\}$, whether e is true of the material (or the world); the true claims are exactly the events of Definition 2.1, $E_D = \{e \in \hat{E}_D : \tau(e) = 1\}$. The truth value may be relaxed to a confidence $\tau(e) \in [0, 1]$.*

A memory that may contain false claims no longer reliably reflects what the agent knows, so the span splits in two:

$$R_{M_D} = \Phi(M_D), \quad R_{M_D}^{\text{good}} = \Phi(M_D \cap E_D). \quad (10)$$

The first is what the memory *believes* it knows; the second, the **good span**, is what it *truly* knows, since it is generated only by true claims. This presumes that Φ is reliable on true inputs; failures of that default belong to the end-to-end layer (Definition 2.6).

The coverage utility of Definition 2.5 ignores truth; where the contrast with a truth-aware variant matters, we denote the coverage utility by u^{cov} .

Definition 4.2 (Precision utility). *The precision utility is*

$$u^{\text{prec}}(q, M_D) = \mathbf{1} \left[q \in \bigcup_{n \in \Phi(M_D \cap E_D)} \mathcal{Q}(n) \right], \quad (11)$$

whether the memory’s good span contains a single knowledge item that answers q .

Proposition 4.1 (Precision is bounded by coverage). *Under Assumption B.2, for every q and M_D , $u^{\text{prec}}(q, M_D) \leq u^{\text{cov}}(q, M_D)$, and therefore, for any write policy π ,*

$$\Delta(\pi) = \mathbb{E}_{q \sim p_D} [u^{\text{cov}}(q, \pi(D)) - u^{\text{prec}}(q, \pi(D))] \geq 0. \quad (12)$$

Proof sketch. Every true claim is a claim, so the good span is a subset of the span and precision can only fall below coverage. The complete proof is given in Appendix D. $\square$

The difference $\Delta(\pi)$ measures how much of a policy’s coverage survives only because the memory stores false claims; we call it the **water-inflation degree** of the policy. A policy whose u^{cov} is high but whose u^{prec} is low reports a water-inflated score whose cost is paid at the end-to-end layer: false knowledge can mislead the reasoner and, because a conflicting claim can overturn previous conclusions when Φ is only locally monotone (Remark B.3, Appendix B), it can even destroy coverage that was previously correct. In this regime, “is there enough information” and “is the information correct” diverge, and coverage ceases to be a proxy for end-to-end correctness (Definition 2.6).

4.2 The Noisy Optimal-Memory Problem

Under noise, the coverage objective is the wrong target: a memory composed mostly of hallucinations can score near-perfect on u^{cov} while answering almost nothing correctly. The optimization should target precision instead.

Definition 4.3 (Noisy optimal memory). *The capacity-constrained optimal memory under noise maximizes the precision utility, reusing the symbol $M_D^*(S)$ of Definition 3.1 for the precision objective,*

$$M_D^*(S) = \arg \max_{\substack{M_D \subseteq \hat{E}_D \\ |M_D| \leq S}} \mathbb{E}_{q \sim p_D} [u^{\text{prec}}(q, M_D)], \quad (13)$$

*and the frontier $U_D^{\text{prec}}(S)$ it induces, the **precision frontier**, is defined analogously to $U_D^*(S)$.*

When Φ is decomposable and the truth function τ is known, this reduces to a weighted maximum coverage in which each claim’s weight is its truth times the query mass it covers (Appendix D): a false claim contributes nothing to u^{prec} yet consumes budget, so the optimum prefers true claims with high coverage.

The decisive difficulty, however, is not the optimization but the estimation. A write policy sees only the noisy claims $\hat{E}_D$, never the truth set E_D ; it must infer which claims are trustworthy. Each claim’s truth $\tau(e)$ is an unobserved latent, and the policy decides, before any query is answered, which claims earn their budget. When a query is answered correctly or wrongly, the precision utility is settled, and that settlement is the only signal the policy receives about τ . Attributing the outcome to the right stored claim is a credit-assignment problem, and learning to write well from this signal is what the sequential formulation of Section 5 makes precise. Section 6 shows the coverage and precision frontiers parting on a concrete noise instance.

Table 1 The progressive taxonomy of memorization settings.

Level	Material $\times$ query	p_D known	Object of learning
0	single, one query	—	none (retrieval)
1a	single, many queries	yes	optimal memory, one-shot
1b	single, many queries	no	p_D , hence the optimal memory, sequential
2	many, many queries	no	transferable write rule π

5 A Problem Setting Taxonomy and Sequential Memorization MDP

The optimality theory of Sections 3 and 4 treats the query distribution p_D and the material distribution $p(D)$ as given. In practice both are unknown, and the only evidence an agent has is the queries it actually faces, disclosed one at a time. This section makes that learning explicit. It organizes the problem settings of memory into a progressive taxonomy, isolates two modes of writing and the delayed-reward structure of the persistent mode, states the agent-level memorization objective, and unifies the settings in a sequential MDP.

5.1 A Progressive Taxonomy of Problem Settings

Memorization settings differ in what is known and what must be inferred. Table 1 summarizes the progression.

Level 0 (single material, single query). Memory is written once and serves a single query. There is no distribution to speak of: the setting is one-shot compression, essentially efficient retrieval from a long material, and it cannot separate memory quality from retrieval quality. It is a degenerate case, useful as a baseline but not as a testbed for memorization.

Level 1 (single material, many queries). Queries against one material arrive repeatedly, and the two cases split on whether the agent knows their distribution. When p_D is known (Level 1a, idealized), the problem is exactly the one-shot optimal-memory problem of Section 3; nothing has to be learned. When p_D is unknown (Level 1b, realistic), each disclosed query is a sample of p_D , and writing must be multi-step: after each query the agent holds one more piece of evidence and can revise the memory, so sequential writing is a stepwise search in memory space that approaches $M_D^*(S)$.

Level 2 (many materials, many queries). Marginalizing over the material distribution turns the object of learning from a per-material memory into a transferable write rule, an agent-level strategy that must work across materials. We state this objective formally in Section 5.2.

The two modes of writing. Write policies come in two modes. **Query-agnostic writing**, $M_D = \pi_{\text{write}}(D)$, fixes the memory before any query and must anticipate the distribution p_D ; this is the persistent form of memory. **Query-aware writing**, $\pi_{\text{write}}(q, D)$, sees the specific query and reduces to per-query compression; for a single query it is long-context retrieval, not memory, because nothing survives for reuse across queries.

The modes and the taxonomy. Query-aware writing is viable only at Level 0, where the memory serves the single query that produced it. From Level 1 up, a memory across queries must be query-agnostic, and the object of learning in Table 1 is exactly what the persistent write must infer: the optimal memory (1a), the distribution (1b), or a transferable rule (2). The persistent mode is hard precisely because a write is tested only by queries that arrive after it, so its value is known when a later query settles the account:

$$\underbrace{\pi_{\text{write}}(D)}_{\text{writing, first}} \longrightarrow \underbrace{u(q, \pi_{\text{write}}(D))}_{\text{settlement, later}}, \quad q \sim p_D. \quad (14)$$

This is a **delayed-reward** structure, present as soon as p_D is unknown (Level 1b) and persisting at every higher level. It is what turns memorization into a learning problem, and it gives rise to two difficulties.

Credit assignment. Whether a query is answered correctly cannot be traced to a specific prior write: the utility depends on the whole memory, which is the cumulative result of many write decisions, so a success or failure cannot be attributed to any one of them.

Exploration–exploitation. To learn which claims are trustworthy (the truth τ under noise), the agent may need to store an uncertain claim and let a later query confirm or refute it. The best long-run memory can require deliberately suboptimal short-run choices.

5.2 A Sequential MDP for Memorization

Levels 1 and 2 share a core that Section 5.1 made explicit: write now, be judged by later queries. Both settings also admit a static form, a single objective over the material distribution: the agent chooses a write rule π , and each material’s memory $\pi(D)$ is judged by the queries it serves.

Definition 5.1 (Agent-level memorization). *The agent-level memorization objective is*

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{D \sim p(D)} \left[\sum_{t=1}^{T_D} \mathbb{E}_{q_t \sim p_D} u(q_t, \pi(D)) \right] \quad \text{s.t.} \quad |\pi(D)| \leq S, \quad (15)$$

where π ranges over write policies, and T_D is the number of queries the memory of material D serves. The memory is a function of the material alone, $\pi(D)$, independent of the queries it serves. Under noise, u is replaced by u^{prec} , and estimating τ becomes part of π .

When π is a parametric model (an LLM with its memory module), this equation is a learning objective, and it is the formal place of “memory construction and learning” in the framework: what is learned is no longer the optimal memory of one material but a rule that generalizes over a distribution of materials.

The relation to the earlier theory is exact. When the material distribution concentrates on a single material, the objective reduces, up to the constant factor T_D , to the capacity-constrained problem of Definition 3.1. An upper bound follows from the per-material frontiers: when all materials share capacity S , no policy can exceed $\mathbb{E}_{D \sim p(D)} [T_D U_D^*(S)]$, the individual frontiers weighted by how many queries each material serves, and the gap between a fixed policy and that bound is its regret.

Real writing, however, is not a single shot: the agent meets materials one at a time, and each write is tested by a later query, so the write rule must be learned online. Modeling the process as a sequential MDP generalizes the write policy from $D \mapsto M_D$ to a sequential update; we overload π_{write} , whose single-argument form is the static write of Section 3.1 and whose four-argument form is the update below.

Definition 5.2 (Sequential memorization MDP). *The process runs in episodes $m = 1, \dots, K$. Each episode draws a fresh material $D_m \sim p(D)$; within it, at steps $t = 1, \dots, T_m$, the agent maintains a memory $M_{m,t}$ with $|M_{m,t}| \leq S$. At each step a query $q_{m,t} \sim p_{D_m}$ is posed, the agent answers $a_{m,t} = \pi_{\text{reasoner}}(q_{m,t}, M_{m,t})$, and the memory updates as*

$$M_{m,t+1} = \pi_{\text{write}}(M_{m,t}, D_m, q_{m,t}, a_{m,t}). \quad (16)$$

At the start of an episode the memory is written from the new material alone, $M_{m,1} = \pi_{\text{write}}(D_m)$, resetting whatever the previous episode built. The memory that answers $q_{m,t}$ was written without seeing it: the query is settled first, and only then does it enter the update. The objective is the expected cumulative utility over episodes,

$$\pi_{\text{write}}^* = \arg \max_{\pi_{\text{write}}} \mathbb{E} \left[\sum_{m=1}^K \sum_{t=1}^{T_m} \gamma^t u(q_{m,t}, M_{m,t}) \right], \quad (17)$$

where u is the coverage utility, replaced by u^{prec} under noise and by u^{e2e} when the end-to-end layer is in view (Definition 2.6). The single-material setting is the degenerate case of one episode, $K = 1$ with D_1 fixed.

Each level is now a specialization of the MDP. Level 0 is the single-step episode ($K = 1, T_1 = 1$); the disclosure timing of Definition 5.2 is relaxed so the query is known before the write. Level 1 is one episode over a fixed material ($K = 1, T_1 > 1$), with the delayed reward of Section 5.1. Level 2 is many episodes over fresh draws $D_m \sim p(D)$ ($K > 1$), each material served by its own sequence of queries.

Table 2 The sequential memorization problem as a Markov decision process.

MDP element	Memory-system meaning
episode	a fresh material $D_m \sim p(D)$, memory reset
state	current memory $M_{m,t}$
action	which claims to write, how to reorganize
reward	utility at query time $u(q_{m,t}, M_{m,t})$
transition	$M_{m,t+1} = \pi_{\text{write}}(M_{m,t}, D_m, q_{m,t}, a_{m,t})$

The MDP reading also makes memory the state representation: the capacity bound $|M_{m,t}| \leq S$ constrains the state space, so a write action is literally the choice of which S claims cover the expected future queries, which is the utility–capacity frontier restated as a control problem. Under noise it closes the estimation loop: settling u^{prec} lets the agent discover that it stored a wrong claim, feeding the estimate of τ , which becomes a trainable object rather than a given.

6 An Illustrative Example: The Utility–Capacity Frontier of Homer’s *Odyssey*

This section turns the machinery of Sections 2–5 into numbers, computing the utility–capacity frontier of one instance explicitly and reading the framework’s concepts off it. The instance is small enough to check by hand and rich enough to exhibit basis and span, the synergy of interacting events, the frontier and its compression zone, the memory–efficiency loss of a write policy, and the divergence of coverage from precision under noise. As throughout, the query distribution is treated as known: the Level 1a setting of Table 1.

The instance. Let D be a summary of Homer’s *Odyssey*, and consider an agent that must answer questions about the epic from memory. The extraction procedure Φ is decomposable, so that the knowledge of a set of events is the union of the knowledge of its members and the coverage of a memory is the union of the per-event answerable sets $S_e = \bigcup_{n \in \Phi(\{e\})} \mathcal{Q}(n)$ (Section 3.3). Six events, one per episode, are extracted, each spanning the atomic facts entailed by its statement, one per query the event answers; these events are the candidate basis of a memory, and Table 3 records the span each contributes.

The query space is eight questions about the epic, with probability masses

$$p_D(q_1, \dots, q_8) = (0.20, 0.05, 0.10, 0.15, 0.05, 0.20, 0.05, 0.20), \quad (18)$$

over the questions q_1 (“How long did Odysseus’s return from Troy take?”), q_2 (“From which city did Odysseus set sail for home?”), q_3 (“Which nymph held Odysseus on her island for seven years?”), q_4 (“Who turned Odysseus’s men into swine?”), q_5 (“Who waited twenty years for Odysseus in Ithaca?”), q_6 (“To which island did Odysseus finally return?”), q_7 (“Why did Poseidon oppose Odysseus’s return?”), and q_8 (“Which figures detained Odysseus on their islands during his wanderings?”). Table 3 lists each event with the items it spans (each tagged with the query it answers) and the query mass the event covers.

Where events interact. The spans of Table 3 are computed event by event, and the frontier below inherits the same choice: every knowledge item is spanned by a single event, and a memory’s span is the union of its events’ items. Decomposability is an assumption about the operator, not a property of the material, and the same events exhibit the alternative: Φ composes events that belong together, so that pairs of events also span items that no single event entails, each answering a further query beyond the eight queries of Table 3:

- e_2 and e_3 together span “Odysseus was detained by enchantresses for eight years” (7+1), answering “For how many years in total did enchantresses detain Odysseus?”;
- e_4 and e_5 together span “Odysseus returned to Penelope after twenty years,” answering “How many years after leaving did Odysseus rejoin his wife?”;
- e_6 and e_1 together span “Poseidon’s wrath made the homecoming last ten years,” answering “Why did the voyage take so long?”.

For each of these pairs, $\Phi(\{e, e'\})$ strictly contains $\Phi(\{e\}) \cup \Phi(\{e'\})$: the span of a memory is more than the union of its parts, and an event’s marginal depends on its companions. If the composite query of the

Table 3 The events of the example, the knowledge items each event spans under the decomposable Φ , and the query mass each covers. The detainer events e_2 and e_3 each span an item answering the detention query q_8 , a deliberate overlap.

event e	stored statement	spans $\Phi(\{e\})$	answers	mass
e_1	“Odysseus spent ten years sailing home from Troy.”	n_a : “the voyage took ten years” (q_1); n_b : “sailed from Troy” (q_2)	q_1, q_2	0.25
e_2	“The nymph Calypso held Odysseus on Ogygia for seven years.”	n_c : “held Odysseus for seven years” (q_3); n_d : “Calypso detained Odysseus” (q_8)	q_3, q_8	0.30
e_3	“Circe kept Odysseus for a year and turned his men into swine.”	n_e : “turned the men into swine” (q_4); n_f : “Circe detained Odysseus” (q_8)	q_4, q_8	0.35
e_4	“The Phaeacians carried Odysseus asleep to the shore of Ithaca.”	n_g : “reached the island of Ithaca” (q_6)	q_6	0.20
e_5	“Penelope, Odysseus’s wife, waited twenty years in Ithaca.”	n_h : “waited twenty years for Odysseus” (q_5)	q_5	0.05
e_6	“Odysseus blinded Polyphemus, the Cyclops son of Poseidon.”	n_i : “blinded the Cyclops, Poseidon’s son” (q_7)	q_7	0.05

Table 4 The optimal memories and frontier values of the example.

S	optimal memory $M_D^*(S)$	$U_D^*(S)$
0	$\emptyset$	0.00
1	$\{e_3\}$	0.35
2	$\{e_3, e_1\}$	0.60
3	$\{e_3, e_1, e_4\}$	0.80
4	$\{e_3, e_1, e_4, e_2\}$	0.90
5	$\{e_3, e_1, e_4, e_2, e_6\}$	0.95
6	$\{e_1, \dots, e_6\}$	1.00

first pair carries mass 0.05 in a variant of the query distribution, then e_3 ’s marginal given e_2 is 0.20: 0.15 for its unique query plus 0.05 for the composite, so composition adds genuinely new coverage. Even so, this marginal falls short of e_3 ’s standalone 0.35, because the overlap on q_8 outweighs the composite’s gain; on this instance marginals still decrease and the greedy guarantee is not threatened. The growing marginal behind Proposition 3.3(b) would need the composite mass to dominate the overlap; Remark C.2 constructs that failure in isolation. What these pairs do exhibit is where multi-hop lives in the framework: a compositional query is answerable from memory only when the composition is already realized inside Φ as a single item.

The frontier. Enumerating the capacity-constrained optima of Definition 3.1 gives Table 4 and the frontier of Figure 2. The marginal gains of successive events in the order of the optimal additions, 0.35, 0.25, 0.20, 0.10, 0.05, 0.05, never increase: each additional event buys no more new coverage than the one before. The frontier saturates at the full-context baseline, $\mathbb{E}_{q \sim p_D}[u(q, E_D)] = 1.00$, reached only at the full material, so the entire regime $S < 6$ is the compression zone, and each shaded slab in Figure 2 is the cost of compression at that capacity.

Reading the frontier. Two features are visible in the numbers. First, the optimum adds events in decreasing marginal value, and greedy selection coincides with it on this instance, which is consistent with, but stronger than, the guarantee of Proposition 3.3(a). Second, the overlap between events shapes the marginals, and at the knowledge level it is an overlap between items: e_2 and e_3 each span an item that answers the detention query q_8 (n_d and n_f in Table 3), so once e_3 is stored, e_2 ’s marginal falls from its standalone mass 0.30 to the

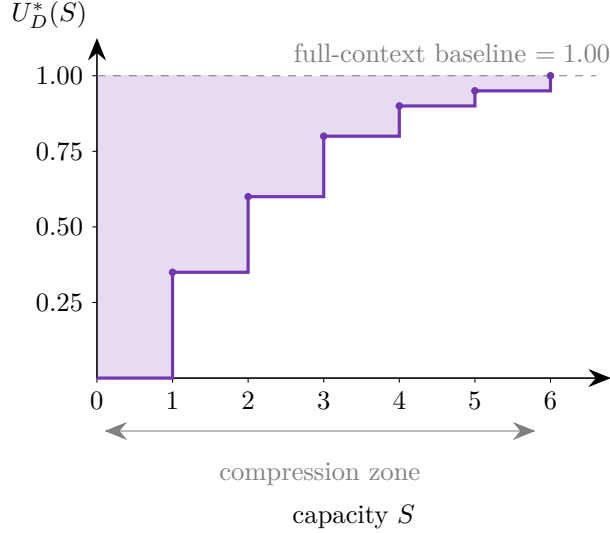


Figure 2 The utility–capacity frontier of the example. The shaded slabs are the cost of compression at each capacity, the gap between the frontier and the full-context baseline; they vanish as the capacity reaches the material size.

mass 0.10 of its unique query; budget spent on an already-covered query is the concrete form of overlap waste.

A write policy on the frontier. The frontier is the yardstick. Consider a policy that prioritizes the wanderings and neglects the homecoming: at capacity $S = 4$ it stores $\{e_3, e_2, e_1, e_6\}$, covering 0.75 (queries $q_1, q_2, q_3, q_4, q_7, q_8$). The frontier at $S = 4$ is 0.90, attained by $\{e_3, e_1, e_4, e_2\}$, so the policy’s memory-efficiency loss is 0.15: it spends the same budget yet leaves the high-mass homecoming query q_6 (0.20) uncovered and wastes space on the already-covered q_8 . This is the sense in which the frontier, rather than any single accuracy number, defines “how good a memory is.”

Under noise. Noise changes what the numbers mean. Suppose extraction also emits a false claim $\hat{e}$: “Odysseus was the prince of Troy who reached the island of Sparta after ten years of wandering,” which answers q_1, q_2, q_6 and carries apparent mass 0.45, but whose truth value is $\tau(\hat{e}) = 0$. The coverage-optimal memory of size 1 is now $\{\hat{e}\}$ (apparent 0.45), beating the true event e_3 (0.35), yet its precision utility is 0: its good span $\Phi(M_D \cap E_D)$ is empty. At size 2 the coverage-optimal $\{\hat{e}, e_3\}$ reports 0.80 apparent coverage but only 0.35 true, a water-inflation degree of $\Delta = 0.45$, while the precision-optimal memory $\{e_3, e_1\}$ scores 0.60 with $\Delta = 0$. The coverage and precision frontiers part wherever the false claim is competitive, and a policy that optimizes u^{cov} inherits the inflation (Proposition 4.1).

7 Related Work

The framework decomposes a memory system into generation, write, and read (Sections 2–5); earlier sections formalized the first two, and we make the read explicit with a lightweight read policy

$$\pi_{\text{read}} : (q, M_D) \mapsto K_q \subseteq \Phi(M_D), \quad (19)$$

the subset of the memory’s spanned knowledge relevant to q ; settlement then uses only the retrieved subset, which recovers the coverage utility of Definition 2.5 as long as the read retrieves every item that covers q . Memory proper is a persistent write coupled with on-demand reading, the RAG-style architecture, whereas a query-aware write for a single query is long-context retrieval rather than memory. We organize the literature along three views: by problem setting, by system component, and by memory representation; recent surveys (Du et al., 2025) map the same space from related angles.

Agent Memory Problem Settings. The taxonomy of Section 5.1 places existing evaluation settings on a progression, summarized in Table 5. Most long-context benchmarks sit at Level 0, the single-material,

Table 5 Evaluation settings in the literature positioned on the taxonomy of Section 5.1.

Level	Defining feature	Representative work and evaluation
0	one material, one query	needle-in-a-haystack (Kamradt, 2023); LongBench (Bai et al., 2024); MemAgent (Yu et al., 2026); ReMemR1 (Shi et al., 2025)
1a	one material, many queries; known p_D	none (theoretical idealization)
1b	one material, many queries; unknown p_D	LoCoMo (Maharana et al., 2024); LongMemEval (Wu et al., 2025); DialSim (Kim et al., 2024); Mem-Gallery (Bei et al., 2026); MemoryAgentBench (Hu et al., 2026); Memory-R1 (Yan et al., 2025)
2	many materials; the write rule generalizes across materials (Definition 5.1)	emerging: MemoryArena (He et al., 2026); AMA-Bench (Zhao et al., 2026); Mem2ActBench (Shen et al., 2026)

single-query case: LongBench (Bai et al., 2024) and needle-in-a-haystack tests (Kamradt, 2023) ask whether information survives inside a long input, and because a single query makes the write query-aware, the optimal memory degenerates toward storing everything; these settings measure efficient retrieval rather than memory. MemAgent (Yu et al., 2026) and ReMemR1 (Shi et al., 2025) train their writes with reinforcement learning yet evaluate one-shot, and the framework locates them at Level 0 despite the learning machinery.

The closest testbeds to the disclosure setting of Level 1b are LoCoMo (Maharana et al., 2024) and LongMemEval (Wu et al., 2025), long multi-session conversations probed across many questions; DialSim (Kim et al., 2024), Mem-Gallery (Bei et al., 2026), and MemoryAgentBench (Hu et al., 2026) extend the setting to multi-party dialogue, multimodal conversation, and incremental multi-turn streams. Most systems evaluated there write the memory once and never update it in response to the questions, so the sequential disclosure of p_D goes unused; Memory-R1 (Yan et al., 2025) is the contrast, training its write policy on LoCoMo and making exactly the disclosure-driven updates those static systems omit, hence its evaluation at Level 1b rather than the one-shot Level 0 of MemAgent and ReMemR1. A recent study on LoCoMo (Terranova et al., 2025) finds that memory-augmented systems cut token use by over 90% at competitive accuracy, direct evidence that memory buys most of the full-context utility at a fraction of the cost.

Level 1a still has no direct evaluation. Level 2 evaluations are just emerging: MemoryArena (He et al., 2026), AMA-Bench (Zhao et al., 2026), and Mem2ActBench (Shen et al., 2026) reuse memory across interdependent tasks, domains, and tool use, approaching the cross-material setting though persistent systems remain largely tested on single corpora.

System Components. Where existing systems differ most is in which of the framework’s components they instantiate (Sections 2–5). Table 6 maps representative persistent systems onto the generation operator Φ , the write policy π_{write} , and the read policy π_{read} .

The generation operator fixes what the stored events span, and the field’s differences lie in how that span is organized: knowledge graphs (HippoRAG, whose successor HippoRAG 2 (Jiménez Gutiérrez et al., 2025) recasts retrieval itself as non-parametric memory), recursive summary trees (RAPTOR (Sarhi et al., 2024), MemWalker (Chen et al., 2023)), reflective summaries (Generative Agents, Reflexion), dialogue consolidation (MemoryBank), and linked note networks (A-MEM).

The write component decides what to store, and the field splits between heuristics and learning. Generative Agents score by recency and importance, Mem0 edits memories through explicit add, modify, and delete operations, MemoryBank applies a forgetting schedule, and MemGPT and MemOS manage a paged store with eviction. MemAgent, ReMemR1, and Memory-R1 (Yan et al., 2025) instead train the write decision with reinforcement learning, instantiating the sequential MDP of Section 5.2, the framework’s own learning target.

The read component decides what the query sees. Every system in Table 6 implements the read policy of this section, through similarity, weighted relevance, graph ranking, or paging, each a way of selecting the subset $K_q \subseteq \Phi(M_D)$ the query needs.

A fourth component acts after the read: Reflexion retries from a written reflection, and ReMemR1 trains the model to reason over its memory; both operate at the end-to-end layer u^{e2e} of Definition 2.6, which the

Table 6 Representative persistent memory systems mapped onto the framework’s components.

System	Memory unit	Generation Φ	Write π_{write}	Read π_{read}
MemoryBank (Zhong et al., 2024)	dialogue summaries	LLM extraction and consolidation	sequential write with forgetting	similarity
Generative Agents (Park et al., 2023)	observation stream	reflective summaries	append, scored by recency and importance	recency, importance, relevance
MemGPT (Packer et al., 2023)	hierarchical pages	LLM summaries	paging and eviction	on-demand paging
MemOS (Li et al., 2025)	unified memory store	LLM and automatic organization	virtual-memory management and eviction	on-demand retrieval and paging
HippoRAG (Jiménez Gutiérrez et al., 2024)	knowledge graph	LLM extraction into a graph	graph store	personalized PageRank
Mem0 (Chhikara et al., 2025)	user facts and preferences	LLM extraction	add, modify, delete rules	relevance
A-MEM (Xu et al., 2025)	linked memory notes	LLM extraction and linking	dynamic note construction	link and similarity
Reflexion (Shinn et al., 2023)	reflective notes	LLM self-reflection	append	similarity over past reflections

framework deliberately separates from coverage.

Memory Representations. The framework models memory as an event set with Φ -generated knowledge and defers the layout of M_D (Section 3). The literature’s representations fall into three families. Explicit symbolic stores keep text items (MemoryBank, Mem0, Generative Agents, Reflexion), closest to the framework’s event set. Structured stores add an index over the symbols, as graphs (HippoRAG), trees (RAPTOR, MemWalker), hierarchical pages (MemGPT, MemOS), or linked note networks (A-MEM), organizing the span and accelerating reading. Parametric stores write memory into the model itself, as memory tokens or recurrent modules learned at test time (MemoryLLM (Wang et al., 2024), Memory3 (Yang et al., 2024), Titans (Behrouz et al., 2025)); these live outside the event-set framing, and the framework leaves M_D open to them.

Whatever the component or representation choices, the framework offers a common yardstick: a memory’s quality is an expected utility and a position on the utility–capacity frontier, so systems designed against ad hoc benchmarks become comparable on a single objective.

8 Discussion and Limitations

8.1 A Research Agenda for Agent Memory

The framework’s payoff is not a particular system but a map of where the unsolved problems live, and the map is organized by the sequential MDP of Section 5.2, which unifies the taxonomy: memory is the state, writing is the action, settlement is the reward, and a fresh material is an episode. Three families of open problems remain, one for the state and actions, one for the reward, and one for the episodes.

Representing memory and the write policy. The MDP’s state is a memory, a set of at most S claims, and its action is the choice of which claims to write; neither is a fixed-dimensional vector, so standard policy parameterizations do not apply. The central question is how to represent a set-valued state and act on it under the capacity bound, and how to learn the generation and read components rather than fix them by hand, since the generation component decides whether multi-hop knowledge is materialized in the state at all.

Learning from delayed reward. The reward is utility at settlement, arriving long after the writing that earned it (Section 5.1); its key questions are credit assignment, attributing a settled outcome to the writes that produced it, and, under noise, trust estimation, learning the truth of stored claims from settlement alone. The sample complexity of this learning, how many settled queries a policy needs to approach the frontier, is open.

Generalizing across materials. The final question is whether a write rule learned on some materials transfers to new ones. Level 2 states the object of learning as a rule over the material distribution (Defini-

tion 5.1), but which parts of it generalize, the extraction and composition operators, the read policy, or the estimation of query distributions, is unknown, and benchmarks for this setting are only emerging (Section 7).

None of these problems is solved by a better storage format. Each is a learning problem on the MDP’s state, reward, and episodes, which is what it means to build memory rather than to design a data structure.

8.2 Limitations

The framework rests on four simplifying choices, and each is a limit of the theory rather than an implementation detail.

Single-item support. Every query is answered by a single knowledge item, and composition is pushed entirely into the generation operator (Assumption 2.1). This is what makes optimal memory a well-defined coverage problem, but it excludes answer-time reasoning: an agent that genuinely combines items to answer a query lies outside the account, and extending the theory would require a joint answer model rather than pure coverage.

Coverage is not correctness. The utility the theory optimizes is coverage, not correctness. The proxy stance of Definition 2.6 treats sufficient information as approximately sufficient for a correct answer; the example supports the stance but does not establish it. Where the reasoner is fallible, or where misleading memory steers it, coverage and end-to-end quality diverge, and the framework’s results do not apply until the end-to-end layer is modeled.

Assumptions on the generation operator. The assumptions on Φ hold only approximately in practice. Real extraction is at best locally monotone, and a conflicting claim can overturn earlier conclusions (Remark B.3, Appendix B); the framework records this failure mode as a research problem rather than resolving it. The clean frontier picture of Section 3.2 depends on monotonicity, and decomposability, which the example assumes, is a special case that real operators meet only roughly.

Representation-agnostic memory. The definition fixes what memory is, a set of events, but not how it is stored, so representation is outside the account: whether a system keeps verbatim records, compressed summaries, graphs, embeddings, or parameters, the theory sees only the events M_D and the operators that map them to knowledge. This is what lets the frontier compare heterogeneous systems on a common yardstick, but the capacity the theory bounds is counted in events, $|M_D| \leq S$, whereas a real memory pays a representation-dependent cost in tokens or storage, and the trade-offs that motivate compressed or parametric memories are set aside. A fuller definition would couple the account to a representation and its cost, or state how that cost depends on the events chosen.

9 Conclusion

The paper set out to give agent memory a definition. A memory is a basis, a subset of a material’s events, and the knowledge it spans under a generation operator is what an agent can draw on; a memory is good insofar as a few events cover as much of the expected queries as the capacity allows. This reduces optimal memory to a coverage problem whose solution traces a utility–capacity frontier, and the frontier is the common yardstick the field has been missing: any system’s memory is a point in the size–utility plane, and its worth is the gap to what is attainable at that size. Under noisy extraction, coverage and precision part company, the water-inflation degree measures how much reported memory quality is bought with false claims, and writing becomes a learning problem of delayed reward, credit assignment, and trust estimation, unified as a sequential MDP in which memory is the state and writing is the action. The framework positions existing systems as differing choices of generation, writing, reading, and reasoning, and it turns the field’s open questions, representing and learning write policies and transferring them across materials, into questions that can now be stated precisely. Whether real systems approach the frontier is an empirical matter the framework does not settle; making that question measurable is the point of the exercise.

References

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2308.14508.
- Ali Behrouz, Peilin Zhong, and Vahab S. Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2025.
- Yuanchen Bei, Tianxin Wei, Xuying Ning, Yanjun Zhao, Zhining Liu, Xiao Lin, Yada Zhu, Hendrik Hamann, Jingrui He, and Hanghang Tong. Mem-Gallery: Benchmarking multimodal long-term conversational memory for MLLM agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.
- Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. Walking down the memory maze: Beyond context limit through interactive reading. arXiv preprint arXiv:2310.05029, 2023.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. arXiv preprint arXiv:2504.19413, 2025.
- Yiming Du, Wenyu Huang, Danna Zheng, Zhaowei Wang, Sebastien Montella, Mirella Lapata, Kam-Fai Wong, and Jeff Z. Pan. Rethinking memory in LLM based agents: Representations, operations, and emerging topics. arXiv preprint arXiv:2505.00675, 2025.
- Zexue He, Yu Wang, Churan Zhi, Yuanzhe Hu, Tzu-Ping Chen, Lang Yin, Ze Chen, Tong Arthur Wu, Siru Ouyang, Zihan Wang, Jiaxin Pei, Julian McAuley, Yejin Choi, and Alex Pentland. MemoryArena: Benchmarking agent memory in interdependent multi-session agentic tasks. In *International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.16313.
- Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in LLM agents via incremental multi-turn interactions. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.05257.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Weijian Qi, Sizhe Zhou, and Yu Su. From RAG to memory: Non-parametric continual learning for large language models. In *International Conference on Machine Learning (ICML)*, 2025. arXiv:2502.14802.
- Gregory Kamradt. Needle in a Haystack: Pressure Testing LLMs. GitHub repository, 2023. <https://github.com/gkamradt/needle-in-a-haystack>.
- Jiho Kim, Woosog Chay, Hyeonji Hwang, Daeun Kyung, Hyunseung Chung, Eunbyeol Cho, Yeonsu Kwon, Yohan Jo, and Edward Choi. DialSim: A dialogue simulator for evaluating long-term multi-party dialogue understanding of conversational agents. arXiv preprint arXiv:2406.13144, 2024.
- Dharshan Kumaran, Demis Hassabis, and James L. McClelland. What learning systems do intelligent agents need? complementary learning systems theory updated. *Trends in Cognitive Sciences*, 20(7):512–534, 2016.
- Zhiyu Li, Chenyang Xi, Chunyu Li, Ding Chen, Boyu Chen, Shichao Song, Simin Niu, Hanyu Wang, Jiawei Yang, Chen Tang, Qingchen Yu, Jihao Zhao, Yezhaohui Wang, Peng Liu, Zehao Lin, Pengyuan Wang, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li, Zhen Tao, Huayi Lai, Hao Wu, Bo Tang, Zhengren Wang, Zhaoxin Fan, Ningyu Zhang, Linfeng Zhang, Junchi Yan, Mingchuan Yang, Tong Xu, Wei Xu, Huajun Chen, Haofen Wang, Hongkang Yang, Wentao Zhang, Zhi-Qin John Xu, Siheng Chen, and Feiyu Xiong. MemOS: A memory OS for AI system. arXiv preprint arXiv:2507.03724, 2025.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of LLM agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. arXiv:2402.17753.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards LLMs as operating systems. arXiv preprint arXiv:2310.08560, 2023.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.

- Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2401.18059.
- Yiting Shen, Kun Li, Wei Zhou, and Songlin Hu. Mem2ActBench: A benchmark for evaluating long-term memory utilization in task-oriented autonomous agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2026.
- Yaorui Shi, Yuxin Chen, Siyuan Wang, Sihang Li, Hengxing Cai, Qi Gu, Xiang Wang, and An Zhang. Look back to reason forward: Revisitable memory for long-context LLM agents. arXiv preprint arXiv:2509.23040, 2025.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- Alessandra Terranova, Björn Ross, and Alexandra Birch. Evaluating long-term memory for long-context question answering. In *NeurIPS 2025 Workshop on Metacognition in Generative AI*, 2025.
- Endel Tulving. Episodic and semantic memory. In Endel Tulving and Wayne Donaldson, editors, *Organization of Memory*, pages 381–403. Academic Press, New York, 1972.
- Yu Wang, Yifan Gao, Xiushi Chen, Haoming Jiang, Shiyang Li, Jingfeng Yang, Qingyu Yin, Zheng Li, Xian Li, Bing Yin, Jingbo Shang, and Julian McAuley. MemoryLLM: Towards self-updatable large language models. arXiv preprint arXiv:2402.04624, 2024.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking chat assistants on long-term interactive memory. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.10813.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. arXiv:2502.12110.
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schütze, Volker Tresp, and Yunpu Ma. Memory-R1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. arXiv preprint arXiv:2508.19828, 2025.
- Hongkang Yang, Zehao Lin, Wenjin Wang, Hao Wu, Zhiyu Li, Bo Tang, Wenqiang Wei, Jinbo Wang, Zeyun Tang, Shichao Song, Chenyang Xi, Yu Yu, Kai Chen, Feiyu Xiong, Linpeng Tang, and Weinan E. Memory³: Language modeling with explicit memory. *Journal of Machine Learning*, 3:300–346, 2024.
- Hongli Yu, Tinghong Chen, Jiangtao Feng, Jiangjie Chen, Weinan Dai, Qiyang Yu, Ya-Qin Zhang, Wei-Ying Ma, Jingjing Liu, Mingxuan Wang, and Hao Zhou. Memagent: Reshaping long-context LLM with multi-conv RL-based memory agent. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2507.02259.
- Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, Yuandong Tian, and Jishen Zhao. AMA-Bench: Evaluating long-horizon memory for agentic applications. arXiv preprint arXiv:2602.22769, 2026.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the 38th AAAI Conference on Artificial Intelligence (AAAI)*, 2024. arXiv:2305.10250.

Appendix Contents

A	Notation	20
B	Assumptions on the Generation Operator	20
C	Optimal Memory: Proofs and Remarks	21
D	Noisy Memory: Proofs and Remarks	22

Appendix

A Notation

The table below collects the notation used throughout the paper, in the order it is introduced.

Events and knowledge	
E, e	event space; an event, an atomic statement about a material
D	a material: a long source of information
E_D	the events contained in material D
N, n	knowledge-item space; a knowledge item, an atomic self-contained unit of information
$\Phi(A)$	generation operator: the knowledge generated by an event set A
N_D	knowledge contained in D : $\Phi(E_D)$
M_D	memory of D : a subset of E_D , the basis
R_{M_D}	span of memory M_D : $\Phi(M_D)$
Queries and utility	
$\mathcal{Q}, q$	query space; a query
$\text{ans}(n, q)$	whether item n alone suffices to answer q
$\mathcal{Q}(n)$	answerable set of n : $\{q \in \mathcal{Q} : \text{ans}(n, q) = 1\}$
$u(q, M_D)$	coverage utility: whether the span of M_D answers q
$u^{\text{prec}}(q, M_D)$	precision utility: whether the good span of M_D answers q
$u^{\text{e2e}}(q, M_D, \pi_{\text{reasoner}})$	end-to-end utility: whether the reasoner answers q correctly
π_{reasoner}	reasoning policy
$p_D(q)$	query distribution for material D
$p(D)$	distribution over materials
Optimal memory and the frontier	
π_{write}	write policy: $D \mapsto M_D \subseteq E_D$
M_D^*	optimal memory: unconstrained maximizer of expected coverage utility
S	capacity bound: $ M_D \leq S$
$M_D^*(S)$	capacity-constrained optimal memory
$U_D^*(S)$	utility–capacity frontier: optimal expected utility at capacity S
S_e	query set of event e : $\bigcup_{n \in \Phi(\{e\})} \mathcal{Q}(n)$
Noise	
$\hat{E}_D$	claims extracted from D ; the true ones form the event set E_D
$\tau(e)$	truth value of claim e ; the true claims are the events
$R_{M_D}^{\text{good}}$	good span: $\Phi(M_D \cap E_D)$
$u^{\text{cov}}(q, M_D)$	coverage utility, written u^{cov} under noise
$\Delta(\pi)$	water-inflation degree of policy π : expected coverage minus precision
$U_D^{\text{prec}}(S)$	precision frontier: the optimal precision utility at capacity S
Settings and the MDP	
π	agent-level write rule
π^*	optimal agent-level write rule: maximizer of the memorization objective
T_D	number of queries served by the memory of material D
K	number of episodes
D_m	material drawn in episode m
$M_{m,t}$	memory at step t of episode m
$q_{m,t}$	query at step t of episode m
$a_{m,t}$	answer produced at step t of episode m
γ	discount factor
Reading	
π_{read}	read policy: $(q, M_D) \mapsto K_q$
K_q	subset of the span $\Phi(M_D)$ retrieved for query q

B Assumptions on the Generation Operator

This appendix states the assumptions on the generation operator in full; Section 2.3 summarizes them in the main text. The operator Φ is an abstract procedure (in practice implemented by an LLM performing extraction, induction, and summarization), and its properties determine the structure of the optimality theory.

Assumption B.1 (Self-containment). *For every event e , $\{e\} \subseteq \Phi(\{e\})$: a single event generates at least its own degenerate knowledge item. This makes the convention $E \subset N$ consistent with $R_{M_D} = \Phi(M_D)$.*

Self-containment is a consistency condition: storing an event never throws the event away. The more substantive assumption is that knowledge grows with the stored set.

Assumption B.2 (Monotonicity). *If $A \subseteq B$ then $\Phi(A) \subseteq \Phi(B)$. More memory is never worse: coverage is non-decreasing in the stored set.*

Monotonicity says that more memory is never worse. Beyond it, the framework can be strengthened or relaxed in specific directions, and we flag the options.

Remark B.1 (Idempotence). *We may additionally assume $\Phi(\Phi(M_D)) = \Phi(M_D)$ (knowledge of knowledge yields nothing new), in which case Φ is a closure operator and the structure aligns with closure systems and formal concept analysis. We do not require this.*

A related question is how knowledge from different events combines. It is tempting to assume that the knowledge of a set is simply the union of the knowledge of its elements, and the following remark explains why we do not.

Remark B.2 (Non-decomposability). *We do not assume that Φ decomposes over events: in general $\bigcup_{e \in M_D} \Phi(\{e\}) \subsetneq \Phi(M_D)$, because cross-event combination can generate knowledge that no single event yields. This is precisely the meaning of pushing composition into Φ (Assumption 2.1). As Section 3 shows, the optimality problem is a monotone set-function maximization in general, and reduces to weighted maximum coverage only in the decomposable special case.*

Finally, real systems meet these assumptions only approximately, and one failure mode matters enough to state explicitly.

Remark B.3 (Real LLMs). *Real extraction procedures are only locally monotone: injecting a fact that conflicts with existing events may overturn prior conclusions, making Φ non-monotonic. This failure mode (harmful memory via conflict injection) is itself a research problem that we do not assume away.*

C Optimal Memory: Proofs and Remarks

This appendix collects the proofs of the propositions in Section 3 and the remarks deferred from the main text.

The unconstrained problem. Section 3.1 works directly with the budgeted problem; for completeness, the unconstrained statement it forgoes is the following.

Definition C.1 (Optimal memory and write policy, unconstrained). *For a material D and a query distribution p_D , the **optimal memory** is any maximizer of the expected coverage utility,*

$$M_D^* = \arg \max_{M_D \subseteq E_D} \mathbb{E}_{q \sim p_D} [u(q, M_D)], \quad (20)$$

*and an **optimal write policy** is any π_{write}^* whose output attains the optimum; a randomized policy may return a distribution over the optimal memories.*

The $\arg \max$ is a set, so optimal memories (and hence optimal write policies) form families of tied maximizers. Under monotonicity this problem is degenerate: storing every event attains the maximum (Proposition 3.1), so the capacity bound is what makes the question non-trivial.

Proof of Proposition 3.1. If $M_D \subseteq M'_D$, then $\Phi(M_D) \subseteq \Phi(M'_D)$ by monotonicity, so the union $\bigcup_{n \in \Phi(M_D)} \mathcal{Q}(n)$ is contained in $\bigcup_{n \in \Phi(M'_D)} \mathcal{Q}(n)$, and the indicator of Definition 2.5 is non-decreasing. $\square$

Proof of Proposition 3.2. The feasible set $\{M_D \subseteq E_D : |M_D| \leq S\}$ grows with S , so the maximum is non-decreasing. For $S \geq |E_D|$ the full set E_D is feasible and, by Proposition 3.1, dominates every other memory. $\square$

Remark C.1 (A non-monotonic signature). *The frontier is non-decreasing only because Φ is monotone. When a conflicting injection can overturn previous conclusions (Remark B.3), more events can lower the utility and $U_D^*(S)$ will dip. Such a dip is the formal signature of harmful memory, and we do not assume it away.*

Proof of Proposition 3.3. Under decomposability, $\Phi(M_D) = \bigcup_{e \in M_D} \Phi(\{e\})$, so the answerable set of a memory is $\bigcup_{n \in \Phi(M_D)} \mathcal{Q}(n) = \bigcup_{e \in M_D} S_e$, and $u(q, M_D) = \mathbf{1}[q \in \bigcup_{e \in M_D} S_e]$. The expected utility is therefore $\mathbb{E}_{q \sim p_D}[u(q, M_D)] = \sum_{q \in \bigcup_{e \in M_D} S_e} p_D(q)$, the total weight of the covered queries: exactly the value of the weighted maximum-coverage instance over the per-event query sets $\{S_e\}_{e \in E_D}$ with element weights $p_D(q)$. NP-hardness follows because a uniform p_D over a finite query set recovers unweighted maximum coverage, a classical NP-hard problem, and the greedy algorithm that repeatedly adds the event covering the most uncovered query mass attains the $(1 - \frac{1}{e})$ -approximation. For part (b), the objective is monotone by Proposition 3.1, and Remark C.2 constructs a monotone but non-decomposable Φ on which the marginal gain grows, so the objective is not submodular, the reduction to maximum coverage fails, and no greedy-style guarantee applies. $\square$

Remark C.2 (Cross-event synergy breaks submodularity). *The general case is genuinely harder. Consider three events a, b, x and four queries $q_1, \dots, q_4$ with uniform distribution. Let $\Phi(\{a\}) = \{n_a\}$, $\Phi(\{b\}) = \{n_b\}$, $\Phi(\{x\}) = \{n_x\}$ with $\mathcal{Q}(n_a) = \{q_1\}$, $\mathcal{Q}(n_b) = \{q_2\}$, $\mathcal{Q}(n_x) = \{q_3\}$, let pair sets be the unions of their singletons, and let $\Phi(\{a, b, x\}) = \{n_a, n_b, n_x, n_{abx}\}$ with $\mathcal{Q}(n_{abx}) = \{q_1, q_2, q_3, q_4\}$. This Φ is monotone but not decomposable. Adding x to $\{a\}$ raises the value by $1/4$, while adding it to $\{a, b\}$ raises it by $1/2$: the marginal contribution grows, so the objective is not submodular. The $(1 - \frac{1}{e})$ guarantee of the decomposable case does not transfer. Whether near-optimal memories can be constructed efficiently under structural assumptions on Φ (for example, bounded cross-event synergy) is an open question.*

D Noisy Memory: Proofs and Remarks

Proof of Proposition 4.1. Since $M_D \cap E_D \subseteq M_D$, monotonicity gives $\Phi(M_D \cap E_D) \subseteq \Phi(M_D)$, so the union of answerable sets in Definition 4.2 is a subset of that in Definition 2.5; the inequality is pointwise, and taking expectations preserves it. $\square$

The decomposable case in detail. When Φ is decomposable and the truth function τ is known, the noisy optimal-memory problem of Definition 4.3 reduces to a weighted maximum coverage in which the weight of a claim e is $\tau(e) \cdot \Pr_{q \sim p_D}[q \in \bigcup_{n \in \Phi(\{e\})} \mathcal{Q}(n)]$, up to overlap between claims: the query mass covered by several chosen claims is counted only once. A false claim contributes nothing to u^{prec} yet consumes budget, so the optimum prefers true claims with high coverage.